

ass9

April 6, 2025

```
[14]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, recall_score, precision_score, f1_score,
    jaccard_score, confusion_matrix, roc_curve, auc
)
```

```
[16]: # Load the data
df = pd.read_csv("C:/Users/cool/Desktop/ML ass/samples_cancer.csv")

# Display basic info
df.head()
```

```
[16]:
```

	ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	\
0	1000025	5	1	1	1	2	1	
1	1002945	5	4	4	5	7	10	
2	1015425	3	1	1	1	2	2	
3	1016277	6	8	8	1	3	4	
4	1017023	4	1	1	3	2	1	

	BlandChrom	NormNucl	Mit	Class
0	3	1	1	2
1	3	2	1	2
2	3	1	1	2
3	3	7	1	2
4	3	1	1	2

```
[18]: # Remove ID column and handle non-numeric issues
df = df.drop("ID", axis=1)
df = df.replace('?', np.nan)
df = df.dropna()
```

```
# Convert to numeric (especially 'BareNuc')
df = df.astype(float)
```

```
[20]: # Features and target
X = df.drop("Class", axis=1)
y = df["Class"]

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
[22]: def evaluate_model(model, kernel_name):
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    y_scores = model.decision_function(X_test_scaled)

    acc = accuracy_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred, pos_label=4)
    prec = precision_score(y_test, y_pred, pos_label=4)
    f1 = f1_score(y_test, y_pred, pos_label=4)
    jacc = jaccard_score(y_test, y_pred, pos_label=4)
    err = 1 - acc
    cm = confusion_matrix(y_test, y_pred)

    print(f"\n--- {kernel_name.upper()} Kernel ---")
    print(f"Accuracy      : {acc:.2f}")
    print(f"Recall        : {rec:.2f}")
    print(f"Precision      : {prec:.2f}")
    print(f"F1 Score       : {f1:.2f}")
    print(f"Jaccard Score  : {jacc:.2f}")
    print(f"Error Rate     : {err:.2f}")
    print("Confusion Matrix:\n", cm)

    fpr, tpr, _ = roc_curve(y_test, y_scores, pos_label=4)
    return fpr, tpr, auc(fpr, tpr), kernel_name
```

```
[24]: kernels = ['linear', 'poly', 'rbf', 'sigmoid']
roc_data = []

for kernel in kernels:
    svm = SVC(kernel=kernel)
```

```
fpr, tpr, roc_auc, label = evaluate_model(svm, kernel)
roc_data.append((fpr, tpr, roc_auc, label))
```

--- LINEAR Kernel ---

```
Accuracy      : 0.97
Recall        : 0.95
Precision     : 0.98
F1 Score      : 0.96
Jaccard Score : 0.93
Error Rate    : 0.03
Confusion Matrix:
[[78  1]
 [ 3 55]]
```

--- POLY Kernel ---

```
Accuracy      : 0.92
Recall        : 0.81
Precision     : 1.00
F1 Score      : 0.90
Jaccard Score : 0.81
Error Rate    : 0.08
Confusion Matrix:
[[79  0]
 [11 47]]
```

--- RBF Kernel ---

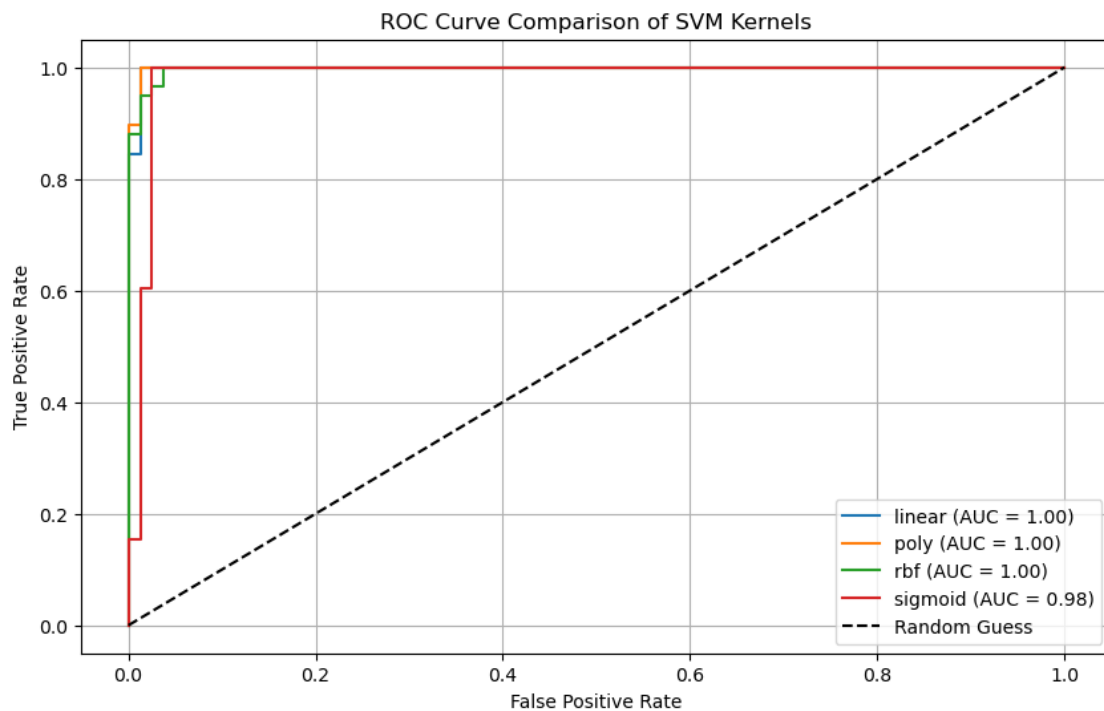
```
Accuracy      : 0.96
Recall        : 0.95
Precision     : 0.96
F1 Score      : 0.96
Jaccard Score : 0.92
Error Rate    : 0.04
Confusion Matrix:
[[77  2]
 [ 3 55]]
```

--- SIGMOID Kernel ---

```
Accuracy      : 0.96
Recall        : 0.93
Precision     : 0.96
F1 Score      : 0.95
Jaccard Score : 0.90
Error Rate    : 0.04
Confusion Matrix:
[[77  2]
 [ 4 54]]
```

```
[26]: plt.figure(figsize=(10, 6))
      for fpr, tpr, roc_auc, label in roc_data:
          plt.plot(fpr, tpr, label=f"{label} (AUC = {roc_auc:.2f})")

      plt.plot([0, 1], [0, 1], 'k--', label='Random Guess')
      plt.xlabel('False Positive Rate')
      plt.ylabel('True Positive Rate')
      plt.title('ROC Curve Comparison of SVM Kernels')
      plt.legend()
      plt.grid()
      plt.show()
```



[]: